

Electronic Companion

Finite-Catalog Renewal Design by Selected-Boundary Resource Paths

Anonymous

EC.1 Purpose and Reading Guide

This companion supplies the retained class-box support details, an explicit dyadic count, the independently implemented certificate arithmetic, and complete computational diagnostics. The principal selected-boundary reduction and capacity-repair proofs are in the article. No empirical calibration is asserted. The historical derivations are preserved in their original files rather than accumulated into this companion as additional prerequisites.

EC.2 The Retained Class-Box Price Identity

Fix a catalog-eligibility class, its last eligible selected level d , normalized weights w_j , and common mean interval $[\ell, u]$. Write $D = \sum_j w_j \min(b_j, d)$ and $c_j = (b_j - d)_+$. Let $\theta_j(s) = \min(b_j, z(s))$ be capped equalization. Set

$$a_j = (\theta_j(\ell) - d)_+, \quad b'_j = (\theta_j(u) - d)_+, \quad q_- = (\ell - D)_+, \quad q_+ = (u - D)_+.$$

When $u \geq D$, the endpoint identities needed for the service correction are

$$\sum_j w_j b'_j = u - D, \quad \sum_j w_j a_j = (\ell - D)_+. \quad (1)$$

Indeed, if $s \geq D$, its water level is at least d , and $\theta_j(s) = \min(b_j, d) + (\theta_j(s) - d)_+$. Summation proves the identity. If $s < D$, the water level is at most d and all positive parts vanish. Thus every independently bounded service vector in $\prod_j [a_j, b'_j]$ has aggregate service in $[q_-, q_+]$. This feasibility inclusion, not merely concavity, is what proves the correction's sign.

For a price λ , define

$$T(\lambda) = \max_{\substack{0 \leq y_j \leq c_j \\ q_- \leq \sum_j w_j y_j \leq q_+}} \sum_j w_j [-h_j(y_j) - \lambda y_j], \quad (2)$$

$$J(\lambda) = \sum_j w_j \max_{a_j \leq y \leq b'_j} [-h_j(y) - \lambda y]. \quad (3)$$

When $u \geq D$, the true class support minus the support over the artificial individual box $[\theta(\ell), \theta(u)]$ is $T - J$. The difference is nonnegative by (1). If $u < D$, both supports are terminal-only and the difference is zero.

Here is the full price-case argument. If $\lambda \leq 0$ and $u \geq D$, positive terminal reward minus price fills all available terminal capacity before service is selected. The true and artificial problems then share terminal reward and its price constant, and their only difference is (2) minus (3). If $\lambda > 0$ and $\ell \leq D$, avoidable service

strictly lowers the priced objective. Both problems have a terminal-only maximizer. At every attainable terminal mean, capped equalization maximizes the common concave interpolant and lies inside the artificial box; hence their suprema coincide. In this case $q_- = 0$, each $a_j = 0$, and both T and J are zero. If $\lambda > 0$ and $\ell > D$, service is unavoidable and both problems choose their smallest permitted mean. The terminal constants agree, and their service difference is again $T - J$. All equalities include $\lambda = 0$, $\ell = D$, $u = D$, zero costs, and zero residual capacities.

For the quadratic service cost, a multiplier ν gives the upper witness

$$T(\lambda) \leq \nu \begin{cases} q_+, & \nu \geq 0, \\ q_-, & \nu < 0 \end{cases} + \sum_j w_j \max_{0 \leq y \leq c_j} \{-\gamma_j y^2 / 2 - (\lambda + \nu)y\}. \quad (4)$$

For positive curvature the scalar maximizer is the stationary point clipped to $[0, c_j]$; for zero curvature one chooses an endpoint or the entire interval when its slope is zero. Equality of this upper witness with a feasible vector verifies T without trusting the allocation routine. The inherited checker also verifies capped equalization, the sign of the correction, and its ownership by exactly one crossing edge or terminal node. Its original two-sided path tables and complete class-box cover remain in the source archive and are rerun for the matched class-subdivision study. They are not used to justify the new resource-path upper bound.

EC.3 An Explicit Dyadic Bound for the Retained Class-Mean Method

The resource-path guarantee does not require class-mean subdivision. We nevertheless give an explicit bound for that retained method, both for comparison and to remove ambiguity about its stopping scale.

Let $X = [\ell, u]$ be a projected class-mean box intersecting $\sum_g W_g s_g = B$. Fix a dependent class g_0 and let $d = E - 1 > 0$. A zero-price box oracle supplies an upper value. Its selected book is feasible at the original promise; bounded one-direction transfers of its class means restore equality. Since every fixed-book class response is L -Lipschitz,

$$\text{zero-price upper value} - \text{recovered lower value} \leq L \sum_g W_g (u_g - \ell_g).$$

Projection onto the root equality gives

$$W_{g_0}(u_{g_0} - \ell_{g_0}) \leq \sum_{g \neq g_0} W_g (u_g - \ell_g).$$

Thus the explicit free-coordinate stopping threshold

$$\eta_{\text{box}} = \frac{\varepsilon}{2L(E - 1)} \quad (5)$$

suffices: if every free weighted width is at most η_{box} , the gap is at most ε . The zero price must be included even when additional prices are used.

Proposition EC.3.1 (Dyadic cover, with the ceiling retained). *Let $w_g^0 = W_g(u_g^0 - \ell_g^0)$ be the initial free weighted widths. Bisect a largest free weighted width at every unresolved node, projecting children onto the equality. Define*

$$q_g = \begin{cases} 0, & w_g^0 = 0, \\ \max\{0, \lceil \log_2(w_g^0/\eta_{\text{box}}) \rceil\}, & w_g^0 > 0. \end{cases}$$

A complete binary cover has at most

$$M \leq \prod_{g \neq g_0} 2^{q_g} \leq \left(\max \left\{ 1, \frac{4L(E-1)}{\varepsilon} \right\} \right)^{E-1} \quad (6)$$

leaves, and at most $2M - 1$ nodes when both children of every split are counted. The first product is a dyadic upper bound, not a claim that projection must generate every tensor-grid leaf.

Proof. A node with gap above ε has some free width greater than η_{box} . Its largest free width also exceeds this threshold. A split halves that coordinate's width, while projection never increases any width. Coordinate g can therefore be split at most q_g times on a root-to-leaf path. Every path has depth at most $\sum_g q_g$, so its binary tree has at most $2^{\sum_g q_g}$ leaves. A full binary tree with M leaves has $2M - 1$ nodes. If infeasible children are pruned, including them before pruning provides the same upper bound.

For the coarse bound, each $w_g^0 \leq 1$. If $\eta_{\text{box}} \geq 1$, all q_g vanish. Otherwise, $2^{\lceil \log_2(1/\eta_{\text{box}}) \rceil} \leq 2/\eta_{\text{box}} = 4L(E-1)/\varepsilon$. Multiplication proves (6). $\square$

For width one and threshold $1/3$, the dyadic requirement is four leaves, not three. The coarse bound must be evaluated with the *same* threshold: (5) then gives $4L(E-1)/\varepsilon = 6$ in one free dimension. Conversely, setting $4L/\varepsilon = 3$ gives $\eta_{\text{box}} = 2/3$ and requires only two leaves. These examples distinguish an omitted ceiling in a putative exact product from a valid coarser bound that already includes its factor of two. They also explain why the stopping threshold must not silently change from $\varepsilon/(2Ld)$ to $\varepsilon/(4Ld)$.

A repaired class-mean net remains valid as well. For each feasible anchor, round $E - 1$ weighted class means down with step $\varepsilon/(2L(E-1))$, set the final coordinate by the equality, clip and repair the remaining deficit by bounded transfers. The weighted absolute perturbation is at most twice the rounded mass. Exact conditional class optimization then loses at most ε . This yields at most $N(1 + 2L(E-1)/\varepsilon)^{E-1}$ calls. Its class-dependent exponent is a property of this particular enumeration, not a lower bound on the joint problem; Theorem 4.2 removes that exponent.

EC.4 Independent Resource-Path Verification

EC.4.1 Marginal intervals and exact common denominators

For one service exit set, all zero-curvature capacity can be used at zero cost. Let that weighted mass be q_0 . For every remaining history, the marginal saturation threshold is $\gamma_j c_j$. Sort distinct thresholds, including zero. On an interval between consecutive thresholds, let S be the saturated set and A the active set. The

mass, reciprocal curvature, and saturated cost are

$$y_0 = q_0 + \sum_{j \in S} \pi_j c_j, \quad R_0 = \sum_{j \in A} \frac{\pi_j}{\gamma_j}, \quad c_0 = \sum_{j \in S} \frac{\pi_j \gamma_j c_j^2}{2}.$$

At required mass y , set $\alpha = (y - y_0)/R_0$. If α lies in the interval, the allocation has $z_j = c_j$ on S , $z_j = \alpha/\gamma_j$ on A , and the cost is $c_0 + R_0 \alpha^2/2$. These first-order conditions support the convex objective globally. At an endpoint, adjacent formulas agree. At total capacity the final formula is valid; a set containing only free capacity has cost zero throughout and needs no active interval.

The new optimizer evaluates capped allocation through the antecedent water-filling routine. The new checker instead builds these marginal intervals directly from the model and evaluates the required mass independently. It recomputes every terminal capacity and every exit set. It checks that the supplied common denominator divides every rational sample exactly and that the integer sample sequence has nonincreasing differences. No floating-point operation is used in the acceptance decision.

The denominator is an exact scaling device, not an accuracy parameter. Its bit length can be large because it is the least common multiple across many distinct rational samples. The polynomial bit bound in the article counts that length explicitly. Reporting only the number of grid states would hide this arithmetic cost.

EC.4.2 Why finite padding preserves the row maxima

Let a previous Bellman vector have finite entries $A_0, \dots, A_R$, and a concave kernel have entries $K_0, \dots, K_C$. Only rows $0 \leq z \leq R + C$ are relevant. Each such row has a feasible column with $0 \leq z - t \leq C$. Set

$$M = \max_t |A_t| + \max_q |K_q|, \quad P = 4M + 1.$$

Extend the kernel to negative indices by $K_0 + Pq$, and beyond C by $K_C - P(q - C)$. These extensions have slopes larger than every internal forward difference on the left and smaller on the right, so the extended sequence remains concave. An infeasible entry is at most $M - P < -M$, whereas a feasible entry is at least $-M$. Thus no infeasible entry is a row maximizer. The extended matrix $A_t + K_{z-t}$ is finite and anti-Monge; its leftmost row maximizers are totally monotone. The checker applies the standard matrix-search reduction of Aggarwal et al. (1987) to this rectangular matrix and verifies that every returned index lies inside the original feasible band.

The optimizer uses a different divide-and-conquer monotone search. Agreement between the two implementations is tested against direct convolution on two thousand seeded arrays with arbitrary previous values and concave kernels. Concavity of the previous Bellman vector is never assumed.

EC.4.3 Coverage and original-space feasibility

For each feasible anchor, the certificate must supply exactly the expected command-count and terminal-vertex state sets. The checker reconstructs all transitions, compares each complete row, computes all terminal completions, and checks the accepted resource band. Missing anchors or shortened rows fail even

if the final displayed bound happens to look plausible. The best score across this complete set determines the upper bound; a purported solver status does not.

The lower witness retains every original history. For each history, the checker verifies nonnegative intermediate service, support inside the selected book, nonnegative probabilities summing to one, every positive-probability realization ceiling, the expected cap, and the reported target. It then checks the exact weighted root promise and subtracts all selected charges once. It validates the interval and requested tolerance using rational arithmetic. Pooling or compressing a display does not remove these individual witnesses.

The corruption suite changes, one at a time, the schema, an anchor, a Bellman row, the upper bound, the lower bound, the best grid score, the common denominator, the promise, the Lipschitz constant, the requested tolerance, service nonnegativity, and a lottery mass. Every mutation is rejected. Assertions implement verification failures; Python must not be invoked with optimization flags that disable assertions. The reproduction scripts enforce ordinary Python execution.

EC.5 Protocol, Models, and Timing Scope

The immutable protocol was committed before the full experiment. Development pilots are disclosed: thirty tiny instances, 1,200 convolution tests, and the first retained difficult instance informed implementation debugging. The new 100-instance regression and 2,000 convolution tests use separate fixed seeds. The difficult-model comparison is deliberately paired with historical cases, not presented as an unseen distribution.

The main study contains 12 difficult-model accuracy requests, 12 strict-prefix cases, 6 controlled eligibility cases, and 14 refinement or perturbation cases. In the strict-prefix family, caps are $j/[2(k+1)]$, costs include zero and positive curvatures, and ceilings differ strictly within the interior gap above $1/2$. Its catalogs are regular grids of 13 and 25 levels. The refinement family supplies nonuniform catalogs. In the eligibility sweep, caps are $j/14$ for twelve histories; grouping their ceilings yields the exact prescribed class counts while keeping probabilities, costs, charges, catalog, budget, and promise fixed. All values are stored as rational strings.

The six difficult models are compared with seven subdivision settings each: four node allowances with the default price setting, two alternative root-price settings at 31 nodes, and individual subdivision at 31 nodes. The original support-indicator mixed-integer formulation is solved with tangent and secant service envelopes. Its total measured time allowance equals the new fine-grid optimization time, except for a minimum of 0.1 seconds per envelope. Every status is retained. This is not a memory-matched comparison; neither a repaired group-net benchmark nor node-local price generation is represented as executed work.

Table EC.1 records all perturbation and refinement cases. Tables EC.2–EC.5 expose the gap-per-node behavior. Tables EC.6 and EC.7 show the alternative root-price settings. Table EC.8 separates optimization, checking, and the exhaustive reference. Complete individual-baseline and mixed-integer records remain in the code-and-data package, including model residuals, probabilities, selected supports, charges, and approximation allowances.

The code uses Python rational arithmetic. The numerical comparison uses SciPy’s mixed-integer interface (Virtanen et al., 2020). The publication environment is pinned and recorded together with the

operating system, package versions, source hashes, and protocol hash. The new study resumes only completed cases under an unchanged protocol. A fresh timing run requires a separate checkout without the generated result directory; it must not overwrite the committed evidence.

EC.6 Preservation and Reproduction

The revision preserves the reviewed scientific tip, both independent reports, and the exact prior root readers. Every inherited source and result remains at its original repository path. The only replaced inherited entries are the current reader entry points and current submission index; their exact predecessor copies are kept in this revision's archive. A manifest compares Git blob hashes against the immutable parent and reports every replacement and its archive location.

The preserved mathematical material includes continuous-design approximations, institutional comparisons, all-promise frontiers, Monge results, resource augmentation, harmonic coarsening, class-mean bounds, and the earlier certificates. Their assumptions and proofs are not altered or silently withdrawn. The current article supplies its own complete proof chain. The preservation map, rather than the introduction, identifies every historical location and explains whether a result is a prerequisite, a retained alternative, or a distinct extension.

To reproduce from the revision root, run the regression tests, the complete fixed study, the reader generator, and the build-and-audit script in that order. A standalone certificate can be verified by calling `code/check_resource.py` on its compressed JSON file. The supplied reproduction archive includes the exact antecedent Python modules imported by the study, along with their paths and hashes. It does not require a network call during verification. Build validation checks page counts, unresolved references and citations, duplicate labels, overfull boxes, the abstract length, and the current mathematical statement inventory. These are production checks, not an editorial acceptance determination.

References

- Aggarwal A, Klawe MM, Moran S, Shor P, Wilber R (1987) Geometric applications of a matrix-searching algorithm. *Algorithmica* 2:195–208. doi:10.1007/BF01840359.
- Hassin R (1992) Approximation schemes for the restricted shortest path problem. *Mathematics of Operations Research* 17(1):36–42. doi:10.1287/moor.17.1.36.
- Lorenz DH, Raz D (2001) A simple efficient approximation scheme for the restricted shortest path problem. *Operations Research Letters* 28(5):213–219. doi:10.1016/S0167-6377(01)00069-4.
- Pagès G, Wilbertz B (2012) Intrinsic stationarity for vector quantization: Foundation of dual quantization. Author manuscript, arXiv:1010.4642v2, March 27.
- Patriksson M (2008) A survey on the continuous nonlinear resource allocation problem. *European Journal of Operational Research* 185(1):1–46. doi:10.1016/j.ejor.2006.12.006.
- Virtanen P, Gommers R, Oliphant TE, et al. (2020) SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods* 17:261–272. doi:10.1038/s41592-019-0686-2.
- Wu X (1991) Optimal quantization by matrix searching. *Journal of Algorithms* 12(4):663–673. doi:10.1016/0196-6774(91)90039-2.
- Zhang H (2012) Solving an infinite horizon adverse selection model through finite policy graphs. *Operations Research* 60(4):850–864. doi:10.1287/opre.1120.1056.

Table EC.1: All catalog-refinement and perturbation requests

Case	E	$E(c)$	Reference	Width	Seconds
gap-low	1	1	0.353281	0.004219	0.124
gap-high	1	1	0.353281	0.004219	0.124
boundary-below	2	1	0.353281	0.004219	0.144
boundary-above	1	1	0.353281	0.004219	0.132
free-0	1	1	0.353281	0.004219	0.124
free-1	2	2	0.360294	0.004081	0.135
free-2	3	2	0.360294	0.004081	0.162
free-3	4	2	0.360294	0.004081	0.175
free-4	5	2	0.360294	0.004081	0.208
dominated-0	1	1	0.353281	0.004219	0.123
dominated-1	2	1	0.353281	0.004219	0.134
dominated-2	3	1	0.353281	0.004219	0.152
dominated-3	4	1	0.353281	0.004219	0.176
dominated-4	5	1	0.353281	0.004219	0.211

All prescribed cases are shown. Dominated insertions have opening charge 3. Their full-catalog class count grows from one to five, while the optimal retained book and value remain unchanged. Free insertion improves the value at the first inserted command and then leaves it unchanged. The near-boundary pair splits classes but need not change the winning book; the main text gives a separate example with an actual value jump.

Table EC.2: Class-mean subdivision: node allowance 1, price steps 4

Seed	Nodes used	Width	Relative	Seconds	Bytes
49500	1	0.003333	1.10%	0.093	1146
49501	1	0.026387	4.93%	0.114	1183
49502	1	0.002996	0.65%	0.097	1106
49503	1	0.002012	6.67%	0.076	967
49504	1	0.002852	0.61%	0.092	1278
49505	1	0.013086	3.00%	0.097	1077

Every interval contains its exhaustive reference. The full JSON also gives oracle calls, rational sizes, normalized errors, and verification times.

Table EC.3: Class-mean subdivision: node allowance 7, price steps 4

Seed	Nodes used	Width	Relative	Seconds	Bytes
49500	7	0.003333	1.10%	0.291	2219
49501	7	0.003887	0.73%	0.351	2994
49502	7	0.002996	0.65%	0.324	2577
49503	7	0.002012	6.67%	0.236	1649
49504	7	0.002809	0.61%	0.322	3227
49505	7	0.011484	2.63%	0.320	2168

Every interval contains its exhaustive reference. The full JSON also gives oracle calls, rational sizes, normalized errors, and verification times.

Table EC.4: Class-mean subdivision: node allowance 31, price steps 4

Seed	Nodes used	Width	Relative	Seconds	Bytes
49500	31	0.002917	0.97%	1.083	5491
49501	31	0.001963	0.37%	1.248	10091
49502	31	0.002234	0.49%	1.268	8818
49503	31	0.001761	5.84%	0.794	3435
49504	31	0.001224	0.26%	1.230	7272
49505	31	0.009043	2.07%	1.221	6317

Every interval contains its exhaustive reference. The full JSON also gives oracle calls, rational sizes, normalized errors, and verification times.

Table EC.5: Class-mean subdivision: node allowance 127, price steps 4

Seed	Nodes used	Width	Relative	Seconds	Bytes
49500	127	0.001771	0.59%	4.143	22271
49501	127	0.001286	0.24%	4.757	45790
49502	127	0.000997	0.22%	4.939	33637
49503	127	0.001509	5.01%	2.918	10087
49504	55	0.000916	0.20%	2.136	11365
49505	127	0.005906	1.35%	4.733	27556

Every interval contains its exhaustive reference. The full JSON also gives oracle calls, rational sizes, normalized errors, and verification times.

Table EC.6: Class-mean subdivision: node allowance 31, price steps 0

Seed	Nodes used	Width	Relative	Seconds	Bytes
49500	31	0.097708	32.34%	0.223	6258
49501	31	0.134337	25.08%	0.312	7641
49502	31	0.063164	13.76%	0.280	6898
49503	31	0.136250	451.88%	0.171	3776
49504	31	0.028245	6.08%	0.258	7185
49505	31	0.170215	38.97%	0.262	5247

This is a root-price sensitivity experiment, not a node-local price-generation experiment.

Table EC.7: Class-mean subdivision: node allowance 31, price steps 8

Seed	Nodes used	Width	Relative	Seconds	Bytes
49500	31	0.002031	0.67%	1.118	6491
49501	31	0.004110	0.77%	1.295	10155
49502	1	0.000094	0.02%	0.138	1092
49503	5	0.000453	1.50%	0.224	1597
49504	1	0.000089	0.02%	0.136	1270
49505	31	0.002040	0.47%	1.281	6948

This is a root-price sensitivity experiment, not a node-local price-generation experiment.

Table EC.8: Optimization, verification, and exhaustive-reference time

Seed	Resource	Checking	Enumeration	States
49500	13.831	2.238	0.113	75,496
49501	18.391	2.862	0.140	94,439
49502	17.073	2.630	0.166	89,574
49503	3.523	0.603	0.083	17,401
49504	15.290	2.263	0.192	80,284
49505	19.762	2.884	0.147	85,604

Times are seconds. Optimization includes exact payoff preprocessing, dynamic programming, book recovery, and table construction. Checking is separately timed. JSON serialization and gzip compression are not in either timer; their byte counts are reported separately. These tiny references are enumerated over books using a separate original fixed-book allocator, not the resource-path code.